

计算机视觉课程课后作业七

建智 2201 谢天赐 20221082044

7.1 完成课堂练习

通过课上学习课下练习，按要求完成代码如下：

```
import cv2
import numpy as np
from tkinter import filedialog
# 鼠标回调函数，绘制点
def MyImageGetMouseEvent_Point(event, x, y, flags, param):
    global myImgGetPointXYName, myImgGetPointXY, myImgMouseClickedPt
    if event == cv2.EVENT_LBUTTONDOWN: # 左键点击
        myFont = cv2.FONT_HERSHEY_SIMPLEX
        w, h = myImgGetPointXY.shape[1], myImgGetPointXY.shape[0]
        cv2.circle(myImgGetPointXY, (x, y), 3, (255, 255, 255), 2)
        cv2.putText(myImgGetPointXY, str((x, y)), (x, y+15), myFont, 0.5, (255, 255, 255), 1)
        cv2.imshow(myImgGetPointXYName, myImgGetPointXY)
        myImgMouseClickedPt.append([x, y])

# 显示 1 张图，点击 4 个点，返回点的位置
def MyImageGetMouseClickedPointData(myImg):
    global myImgGetPointXYName, myImgGetPointXY, myImgMouseClickedPt
    myImgMouseClickedPt = []
    myImgGetPointXYName = "AddPointByClickLeftMouseButton"
    myImgGetPointXY = myImg.copy()
    cv2.imshow(myImgGetPointXYName, myImgGetPointXY)
    cv2.setMouseCallback(myImgGetPointXYName, MyImageGetMouseEvent_Point)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
    print("总点数=" + str(len(myImgMouseClickedPt)) + ", 坐标数据=" + str(myImgMouseClickedPt))
    return myImgGetPointXY, myImgMouseClickedPt

# 建立 2D 透视变换系数求解的矩阵 G 和常数列向量 B
def MyImageCreateMatrixFor2DPerspectiveGB(ptA, ptB):
    G = np.zeros((8, 8), float)
    B = np.zeros((8, 1), float)
    for i in range(4):
        xa, ya = ptA[i]
```

```

xb, yb=ptB[i]
G[2i]=[xa, ya, 1, 0, 0, 0, -xbxa, -xbya]
G[2i+1]=[0, 0, 0, xa, ya, 1, -ybxa, -ybbya]
B[2i]=xb
B[2i+1]=yb
return G, B

#主函数
def main():
    fn=filedialog.askopenfilename(title="请选择图像文件",
    filetypes=[("Imagefiles", ".jpg.png.bmp"), ("Allfiles", ".")])
    if not fn:
        return
    src=cv2.imread(fn)
    if src is None:
        print("无法读取图像， 请检查文件路径。")
        return
    _, ptB=MyImageGetMouseClickPointData(src)
    ptA=[]#初始化 ptA 为空列表
    w, h=src.shape[1], src.shape[0]#确保使用 src 的宽高
    #使用 append 方法添加元素
    ptA.extend([[0, 0], [w-1, 0], [0, h-1], [w-1, h-1]])
    G, B=MyImageCreateMatrixFor2DPerspectiveGB(ptA, ptB)
    G_inv=np.linalg.inv(G)
    K=np.dot(G_inv, B).flatten()
    rst=np.zeros((src.shape[0], src.shape[1], 3), np.uint8)+255
    for i in range(src.shape[0]):
        for j in range(src.shape[1]):
            x, y=j, i
            c=xK[6]+yK[7]+1
            if c!=0:
                xb=int((xK[0]+yK[1]+K[2])/c)
                yb=int((xK[3]+yK[4]+K[5])/c)
                if 0<=xb<src.shape[1] and 0<=yb<src.shape[0]:
                    rst[yb, xb]=src[i, j]
    cv2.imshow("PerspectiveTransformResult", rst)
    cv2.waitKey(0)
    cv2.destroyAllWindows()

if __name__=="__main__":
    main()

```

代码具体任务包括：

1. 确定坐标点 PtA: 确定一个矩形的四个角点坐标，这些点将作为

变换的参考点。

2. 图像处理：打开并显示图像 `src`，获取其宽度 `w` 和高度 `h`。
3. 建立矩阵 `G` 和向量 `B`：根据用户在图像上点击确定的四个点 `PtB`，建立求解变换系数的矩阵 `G` 和向量 `B`。
4. 求解线性方程组：通过求解线性方程组获得变换系数 `K`。
5. 像素扫描与色彩赋值：按行列扫描源图像 `src` 的像素，对每个像素进行透视变换，并在新图像 `rst` 中赋予相应的色彩值。
6. 显示结果：显示经过透视变换后的结果图像 `rst`。

步骤与分析：

1. 图像读取与显示：使用 `cv2.imread` 函数读取图像，并使用 `cv2.imshow` 函数显示图像，以使用户能够选择坐标点。
2. 获取坐标点：通过 `MyImageGetMouseClickPointData` 函数，让用户在图像上点击选择四个点，分别对应于 `PtA` 和 `PtB`。
3. 矩阵 `G` 和向量 `B` 的建立：根据 `PtA` 和 `PtB` 的坐标，使用 `MyImageCreateMatrixFor2DPerspectiveGB` 函数建立矩阵 `G` 和向量 `B`。
4. 求解变换系数 `K`：通过矩阵 `G` 的逆矩阵与向量 `B` 的点积，求解出变换系数 `K`。
5. 透视变换实现：遍历图像的每个像素，使用变换系数 `K` 计算其在新图像中的对应位置，并进行色彩赋值。

运行效果：



图 1

如图 1 所示，程序弹出选中图片，通过鼠标点击的点对应 `pta` 和 `ptb`。输出图片如图二所示。



图 2

7.2 研究分析

问题 1:

课本计算方法代码实现如图所示:

```
def MyImageCreateMatrixFor2DPerspectiveGB(ptA, ptB):  
    # 创建矩阵 G 和向量 B  
    G = np.zeros((8, 8), float)  
    B = np.zeros((8, 1), float)  
  
    k = -1  
    for i in range(4):  
        k += 1  
        xa, ya = ptA[i][0], ptA[i][1]  
        xb, yb = ptB[i][0], ptB[i][1]  
  
        G[k, 0], G[k, 1], G[k, 2] = xa, ya, 1.0  
        G[k, 3], G[k, 4], G[k, 5] = 0.0, 0.0, 0.0  
        G[k, 6], G[k, 7], B[k] = -xb * xa, -xb * ya, xb  
  
        k += 1  
        G[k, 0], G[k, 1], G[k, 2] = 0.0, 0.0, 0.0  
        G[k, 3], G[k, 4], G[k, 5] = xa, ya, 1.0  
        G[k, 6], G[k, 7], B[k] = -yb * xa, -yb * ya, yb  
  
    return G, B  
# 手动计算矩阵和结果  
def manual_perspective_transform(src, ptA, ptB):
```

图 3

运算结果对比如图 4 所示

```
C:\Users\86182\anaconda3\envs\Y0L0v8-Deepsort\python.exe C:\Users\86182\Desktop\计算机视觉作业\作业7\1.py  
OpenCV 计算的透视变换矩阵 G:  
[[ 0.90925756  0.          50.          ]  
 [ 0.          0.86363636 50.          ]  
 [-0.          -0.          1.          ]]  
手动计算的透视变换矩阵 G:  
[[ 9.09257562e-01 -3.41523684e-18 5.00000000e+01]  
 [ 0.00000000e+00 8.63636364e-01 5.00000000e+01]  
 [ 0.00000000e+00 6.09863722e-20 1.00000000e+00]]  
  
进程已结束,退出代码0
```

图 4

比较结果:

1. 矩阵形式: 两个矩阵在形式上非常相似, 都是 3x3 的矩阵, 且第三行第

三列的元素都是 1，这符合透视变换矩阵的标准形式。

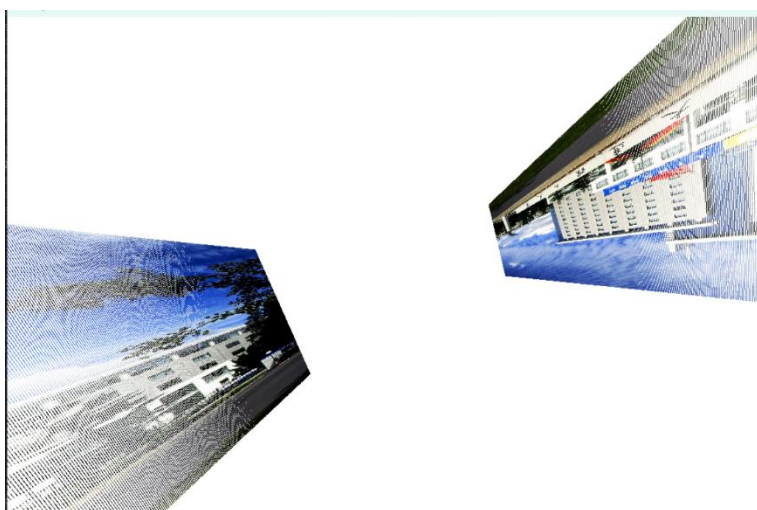
2. **矩阵元素：**矩阵的元素在数值上也非常接近，但存在一些微小的差异。这可能是由于计算方法的不同或者数值精度的差异导致的。
3. **数值精度：**手动计算的矩阵中包含了非常小的数值（如 $-3.41523684e-18$ 和 $6.09863722e-20$ ），这些可能是由于浮点数运算的精度问题。而 OpenCV 计算的矩阵则没有这些极小的数值，这可能是因为 OpenCV 在内部处理时进行了四舍五入或者截断。

分析：

4. **数值差异的原因：**手动计算的矩阵和 OpenCV 计算的矩阵之间的差异可能是由于不同的数值计算方法或者不同的数值稳定性策略。OpenCV 作为一个成熟的图像处理库，其内部实现通常会经过优化，以确保计算的稳定性和效率。
5. **应用影响：**尽管两个矩阵在数值上有所不同，但它们在实际应用中可能产生非常相似的变换效果。这是因为图像处理通常对小的数值变化不敏感，尤其是在像素级别的操作中。
6. **精度和稳定性：**在实际应用中，选择哪个矩阵可能取决于对精度和稳定性的需求。如果需要更高的数值稳定性，可能会倾向于使用 OpenCV 的实现。如果需要完全控制计算过程，可能会选择手动计算。

问题 2：

透视角度较大的时候会出现如图所示的空白点，分析得到以下几种区域：



1. **坐标映射超出图像边界：**在透视变换中，源图像中的每个像素点会被映射到新图像中的一个新位置。如果这个新位置超出了新图像的边界，那么这个像素点在新图像中就没有对应的位置，从而在新图像中形成空白点。
2. **非整数坐标：**由于图像的像素坐标必须是整数，如果透视变换计算出的坐标不是整数，那么在映射过程中可能会丢失一些像素点，导致空白点的出现。
3. **透视变换矩阵不准确：**如果透视变换矩阵的计算不准确，可能会导致源图像中的一些区域在变换后没有正确映射到目标图像中，从而产生空白点。
4. **图像插值方法：**在进行透视变换时，如果使用了不当的图像插值方法，可能会导致图像质量下降，出现空白点或模糊区域。

考虑本方法，其主要原因可能是坐标非整数以及未使用相关图像插值方法

来应对此情况。

容易出现较多空白点的区域为：

1. **图像边缘：**由于透视变换可能会将源图像的边缘区域映射到目标图像的外部，因此在目标图像的边缘区域可能会出现较多的空白点。

2. **源图像的角落：**如果源图像的角落在透视变换后映射到目标图像的外部，那么目标图像的对应区域可能会出现空白。

3. **源图像的中心区域：**如果透视变换的参考点（即用户选择的四个点）没有覆盖源图像的整个区域，那么源图像的中心区域在变换后可能会映射到目标图像的外部，导致空白。

4. **非线性变换区域：**透视变换是一种非线性变换，它可能会导致源图像中的一些区域在变换后被拉伸或压缩，从而在目标图像中形成空白区域。

6. **图像的对角线方向：**透视变换可能会将源图像的对角线方向拉伸，导致对角线方向的区域在变换后出现空白。

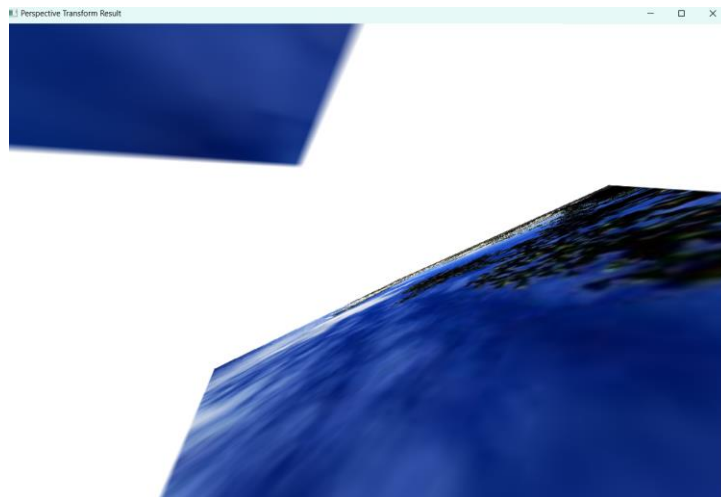
为了确定哪些区域会出现较多的空白点，可以观察源图像和目标图像的对应关系，以及透视变换矩阵的具体参数。通过分析这些因素，可以预测哪些区域在变换后可能会出现空白，并采取相应的措施来减少或消除这些空白点。

问题 3:

解决方式为将透视变换引入**双线性插值方法**：

`cv2.warpPerspective` 函数使用插值方法来估计变换后图像中非整数坐标的像素值。常用的插值方法包括最近邻插值、双线性插值等。插值方法可以平滑图像，减少由于像素坐标不匹配导致的锯齿效应或黑边。

结果如图所示：



问题 4:

“鸟瞰图”变换主要是使用了**逆透视变换方法**。

逆透视变换是一种图像处理技术，用于纠正由于相机视角导致的图像透视失真。其基本原理是通过找到一个合适的变换矩阵，将图像从一个透视视角变换到一个正交视角，从而消除透视效果，使图像看起来像是从正面拍摄的。

方法原理

逆透视变换的核心是找到从图像平面到相机平面的映射关系。这通常涉及到相机的内参（焦距、主点等）和外参（相机的位置和旋转）。通过这些参数，

我们可以构建一个从图像平面到相机平面的投影矩阵，然后求其逆矩阵，得到从相机平面到图像平面的逆透视变换矩阵。使用这个逆矩阵，我们可以将图像中的点映射回相机平面，从而消除透视效果。

以下是一个使用 Python 和 OpenCV 库进行逆透视变换的简单示例：

```
```python
import cv2
import numpy as np

读入图片
img = cv2.imread('input.jpg')

原图中的四个角点 pts1（左上、右上、右下、左下），与变换后矩阵位置 pts2
pts1 = np.float32([[56, 65], [368, 52], [28, 387], [389, 390]])
pts2 = np.float32([[0, 0], [300, 0], [300, 300], [0, 300]])

生成透视变换矩阵；进行逆透视变换
M = cv2.getPerspectiveTransform(pts2, pts1) # 注意这里使用的是 pts2 到 pts1 的变换
dst = cv2.warpPerspective(img, M, (300, 300))

显示图片
cv2.imshow('input', img)
cv2.imshow('output', dst)
cv2.waitKey(0)
cv2.destroyAllWindows()
```
```

与问题三中使用的双线性插值的主要区别

逆透视变换

目的： 逆透视变换主要用于校正图像中的透视失真，使得图像看起来像是从正面拍摄的。这种变换通常用于将图像从一个透视视角变换到一个正交视角。

原理： 逆透视变换通过找到一个变换矩阵，将图像中的点从透视视角映射到一个平面上，从而消除透视效果。这通常涉及到相机的内参和外参，通过这些参数构建一个从图像平面到相机平面的投影矩阵，然后求其逆矩阵，得到从相机平面到图像平面的逆透视变换矩阵。

应用场景： 逆透视变换常用于图像校正、计算机视觉中的物体识别和测量等场景。

双线性插值透视变换

目的： 双线性插值透视变换主要用于图像的几何变换，如旋转、缩放、倾斜等，同时保持图像的透视效果。

原理： 双线性插值透视变换通过构建一个透视变换矩阵，将图像中的点从一个视角变换到另一个视角。在变换过程中，使用双线性插值方法来计算新位

置的像素值，以保持图像的连续性和平滑性。

应用场景： 双线性插值透视变换常用于图像编辑、图像缩放、图像旋转等场景，以及在保持透视效果的同时进行图像的几何变换。

主要区别

变换目的不同：

逆透视变换旨在消除透视失真，使图像看起来更真实。

双线性插值透视变换旨在在保持透视效果的同时进行图像的几何变换。

变换矩阵不同：

逆透视变换的矩阵是通过相机参数计算得到的，用于将图像从透视视角映射到平面视角。

双线性插值透视变换的矩阵是根据变换需求（如旋转角度、缩放比例）计算得到的，用于在保持透视效果的同时进行图像的几何变换。

插值方法的应用：

在逆透视变换中，双线性插值通常作为一种可选的插值方法，用于在变换过程中计算新位置的像素值。

在双线性插值透视变换中，双线性插值是核心的插值方法，用于在变换过程中保持图像的连续性和平滑性。

结果图像的不同：

逆透视变换的结果图像消除了透视失真，看起来更接近于正交视角拍摄的图像。

双线性插值透视变换的结果图像保留了透视效果，但图像的尺寸或视角可能发生了变化。